

Fine-grained Role-Based Access Control in Data Lakes by Encryption

Object Store Wrapper Service (OSWS)

Andreas Severin Hauch Trøstrup ○ Lucas Frey Torres Hanson

IT University of Copenhagen

Supervisor: Martin Hentschel

June 11th 2026

Agenda

• Introduction	3
• Related Work	4
• System Design	5
▶ Architecture Overview	6
▶ Design Choices	7
▶ Examples	11
• Methodology	13
▶ Micro-benchmarks	14
▶ E2E-benchmarks	15
• Results	16
• Discussion	17
▶ Limitations and future work considerations	18
• Conclusion	19

Introduction

- In data lakes, compute and storage are separate.
- Object Store only understands *files* $\Rightarrow$ coarse access control

Option 1: Cloud Platform

- Fine-grained access control
- Vendor lock-in
 - “fully managed all-in-one cloud platform”

Option 2: Vended Credentials

- File-level access control
 - Coarse access control
- Use preferred query engine

Solution

- OSWS
 - It decrypts columns based on authorization

Related Work

- “Membrane: A Cryptographic Access Control System for Data Lakes”
 - Cell-level access control
 - Pull the whole file, but operate only on the view
 - Client is responsible for providing keys/views
- “One Stone, Three Birds: Finer-Grained Encryption with Apache Parquet @ Large Scale” (Uber paper)
 - Uses standard Parquet files and Parquet Modular Encryption (PME)
 - Uses a standard KMS for cryptographic keys
 - Client uses Parquet library extension to read and write

Them vs OSWS

- Them: Reader/writer needs to be modified
- OSWS: Want to ensure no reader/writer modifications
 - Previously: Wanted option for local decryption

System Design

- Wrap S3-compatible Object Store
- Provide S3-compatible API
- Enforce column-level Role-Based Access Control with Parquet Modular Encryption
- DuckDB, Spark, Python – should be able to use OSWS with no modifications

System Design: Architecture Overview

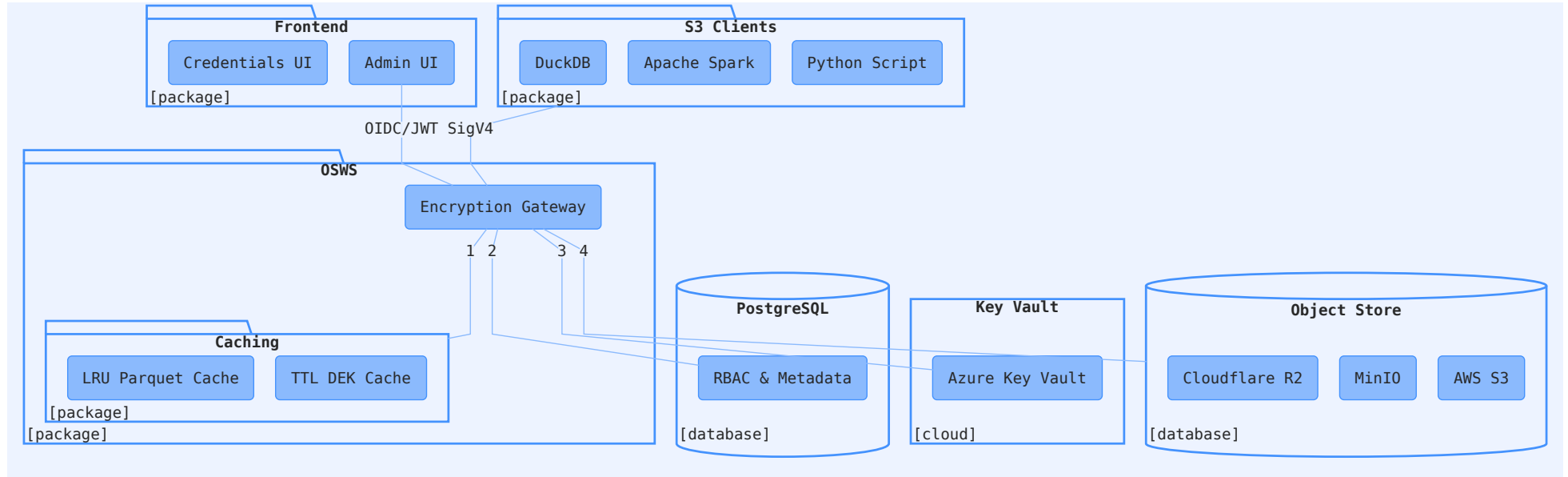


Figure 2: Architecture of OSWS showing components, external services, and communication paths
1 referencing looking up cached DEKs and Parquet files | 2 referencing RBAC calls | 3 referencing DEK wrap or unwrap | 4 referencing S3 API calls

System Design: Design Choices - Database Design + RBAC

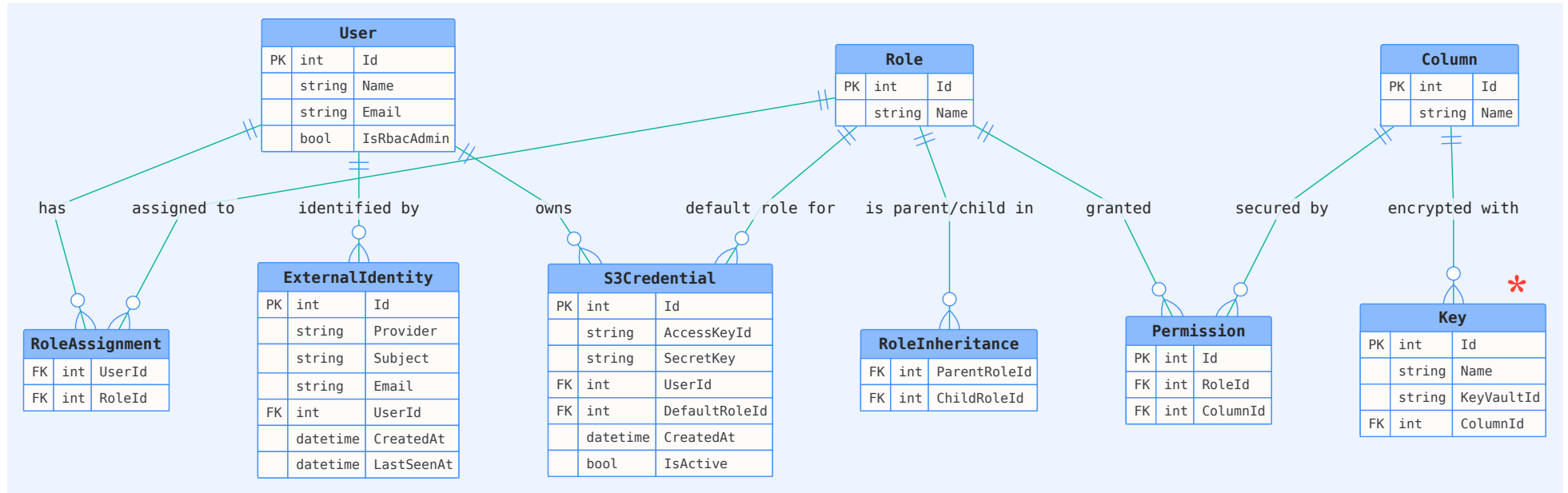


Figure 3: Entity Relationship Diagram showing the design of the database

- S3 API Credentials, Identity- & RBAC-metadata

System Design: RBAC Example

- Many-to-many: User $\leftrightarrow$ RoleAssignment $\leftrightarrow$ Role
- Many-to-many: Role $\leftrightarrow$ Permission $\leftrightarrow$ Column
- Role hierarchy via recursive SQL (self-referential RoleInheritance table)

Example:

Role *Analyst* $\rightarrow$ direct access to columns: name, age

Role *Admin* $\rightarrow$ direct access to column: ssn

Admin inherits *Analyst* $\rightarrow$ Admin sees all columns.

System Design: Design Choices - Frontend

- Credentials and RBAC managed through frontend
 - Interacts with API
- Users log in via OIDC
- Admins manage RBAC with SQL-like grammar

```
CREATE ROLE admin;
=> POST /api/admin/roles { name: "admin" }
CREATE ROLE intern;
=> POST /api/admin/roles { name: "intern" }
GRANT intern TO ROLE admin;
=> POST /api/admin/roles/1/inherit/2
GRANT ACCESS ON name TO intern;
=> POST /api/admin/columns/1/roles/1
GRANT ACCESS ON ssn TO admin;
=> POST /api/admin/columns/2/roles/2
GRANT admin TO USER alice;
=> POST /api/admin/users/1/roles/1
```

Listing 1: Example statements and their parsed API calls

System Design: Design Choices - Envelope Encryption

- Original idea did not include envelope encryption
- KV does not allow for fetching cryptographic keys
- Decrypting columns inside KV is extremely slow
- Allows for caching unwrapped DEKs

System Design: Design Choices - Caching

DEK Cache

- Store unwrapped DEKs in in-memory cache
 - Results in less KV calls and less latency
- The unwrapped DEK gets stored with a unique identifier which also references the KEK identifier
- Uses predefined TTL

Encrypted File Cache

- Cache full encrypted files to skip Object Store
- LRU Policy

System Design: Walkthrough Examples - Encryption Flow

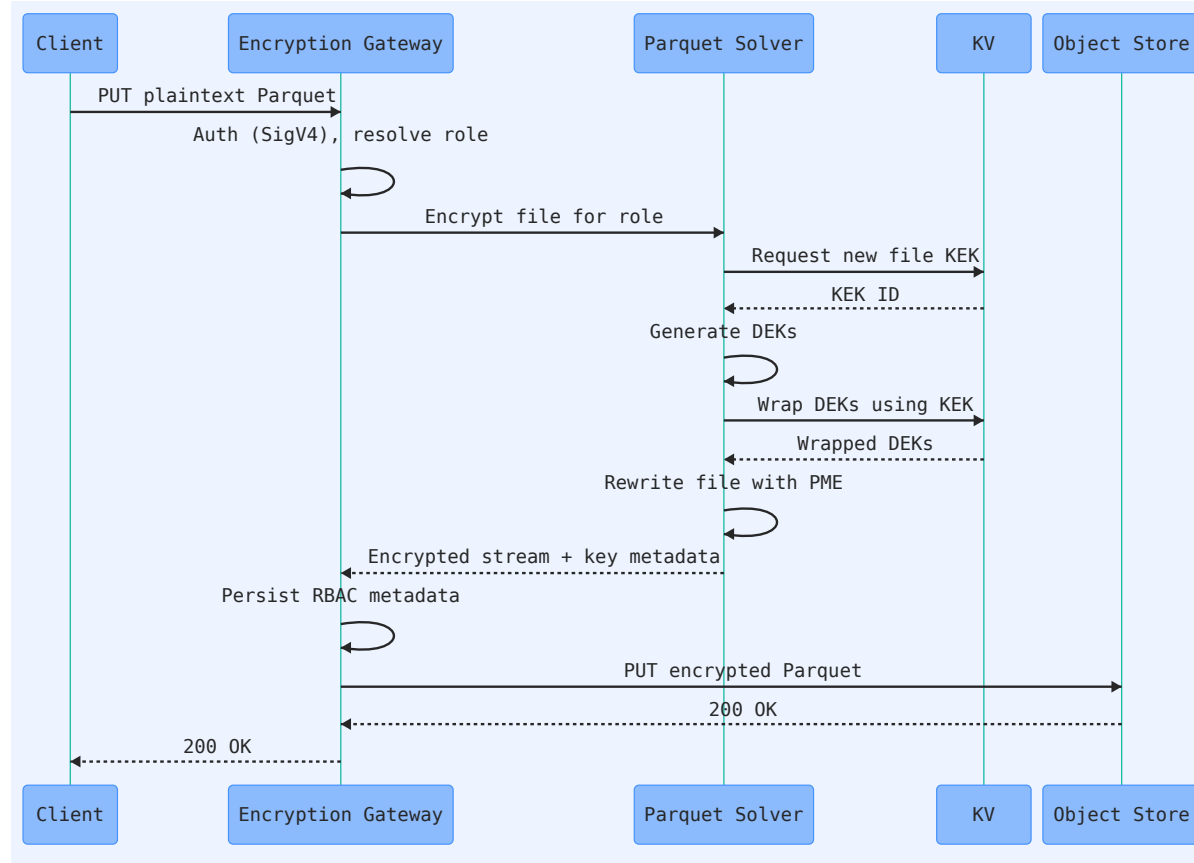


Figure 4: Encryption flow of OSWS

System Design: Walkthrough Examples - Decryption Flow

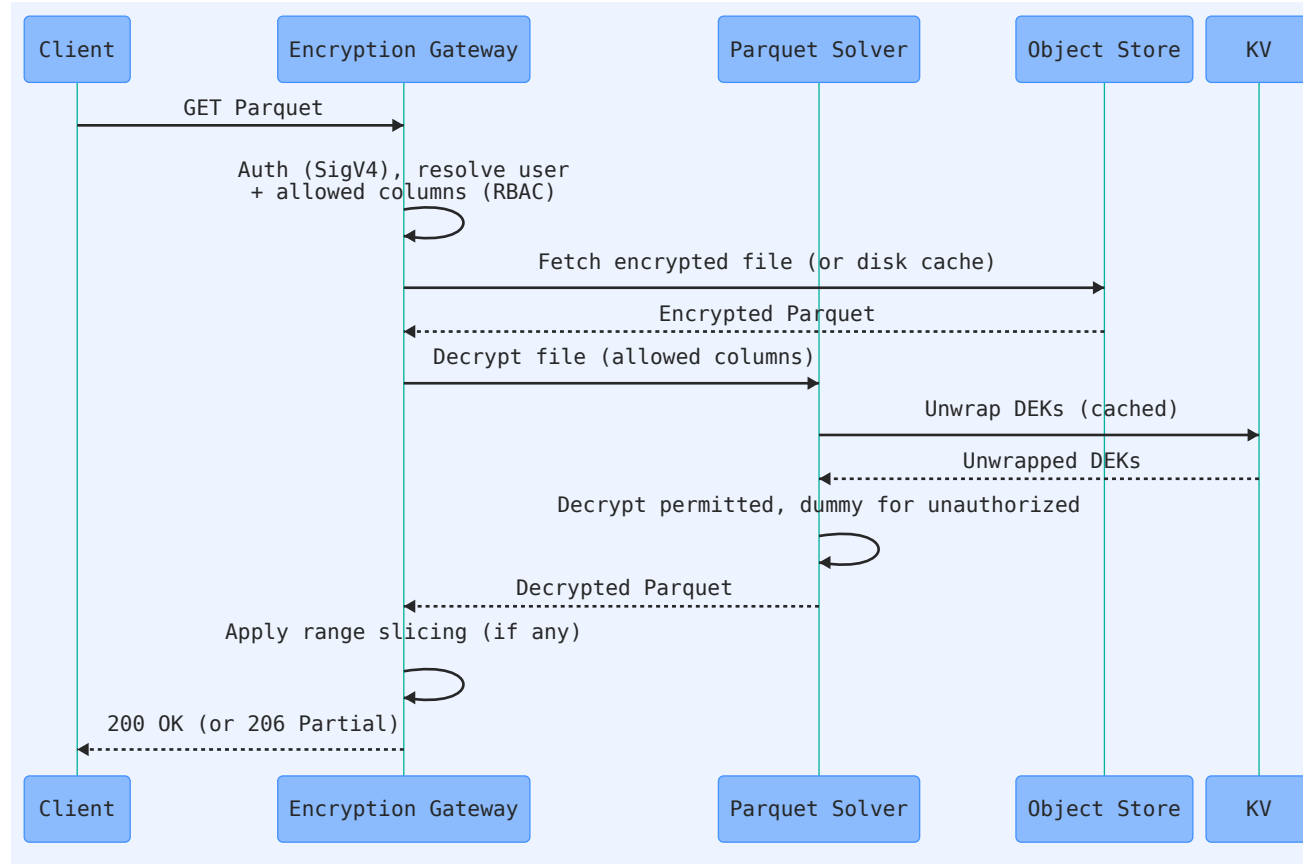


Figure 5: Decryption flow of OSWS

Methodology

- All benchmarks run on Digital Ocean VM (8 Core AMD, 16GB DRAM)
Digital Ocean Space
- Micro-benchmarks
 1. Permission Service
 2. Permission Hierarchy
 3. Unwrapping
 4. Decryption
- E2E-benchmarks
 1. S3
 2. OSWS +encryption +cache
 3. OSWS +encryption -file cache
 4. OSWS +encryption -DEK cache
 5. OSWS -encryption

Category	Benchmark	Configuration	Measurements
E2E	S3 (Spaces)	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)	Latency
E2E	OSWS +encryption +caching	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)*	Latency
E2E	OSWS +encryption -file caching	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)*	Latency
E2E	OSWS +encryption -DEK caching	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)*	Latency
E2E	OSWS -encryption	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)*	Latency
Micro	Permission Service	No. flat hierarchy roles: 4, 64, 256	Latency
Micro	Permission Hierarchy	No. hierarchical depth roles: 0, 4, 16, 64	Latency
Micro	Unwrapping	Key size: 128, 192, 256 bits	Latency
Micro	Decryption	No. rows: 1,000, 10,000, 250,000, 1,000,000, 2,000,000 (50 columns)	Latency

Table v: Main configurations of benchmarks that are to be run

Results: Micro-Benchmarks

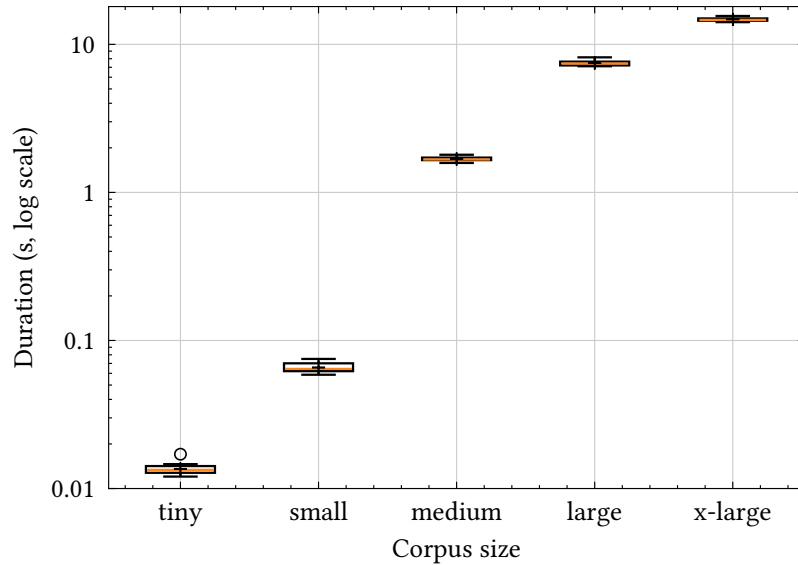


Figure 6a: Decryption boxplot distribution per parameter (logarithmic y axis)

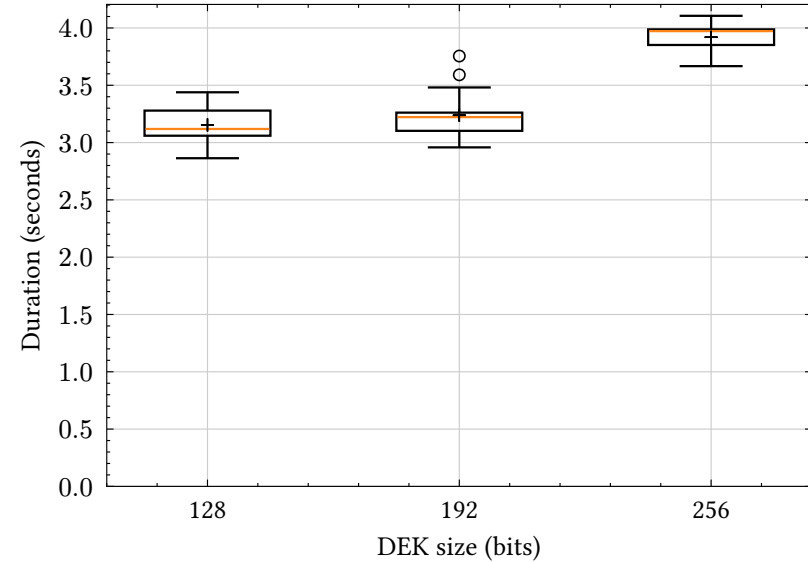


Figure 6b: KeyUnwrap box-plot distribution per parameter set (tiny corpus only; linear y axis)

- Roles had minor contribution to latency ($\sim 3 - 30\text{ms}$)
- Main Bottleneck
 - DEK unwrap (fixed KV cost)
 - Decryption (scales with file size).

Results: E2E Latency

- DEK cache is essential (500MB + timeout)
- GET warm:
 - Tiny: 30 ms
 - Medium: ~ 1.6 vs ~ 0.5 seconds

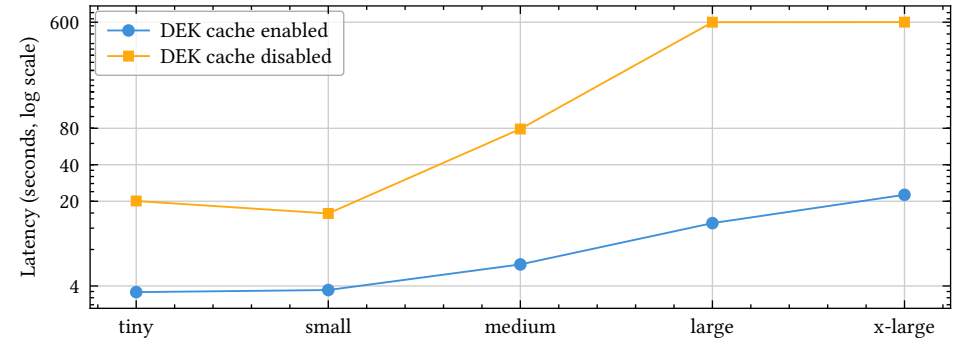


Figure 7 DEK cache impact on median cold GET latency. (logarithmic scale y axis)

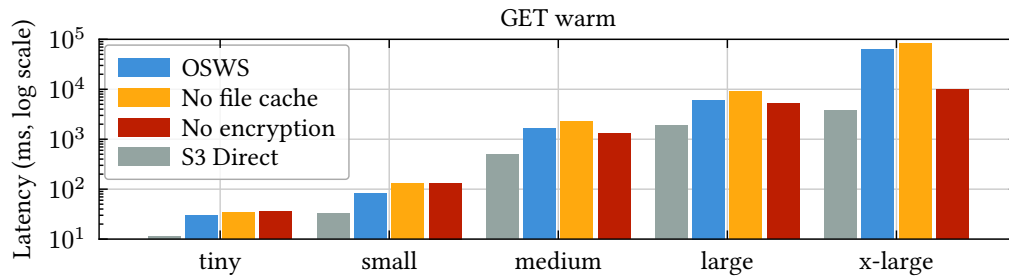


Figure 8a Median GET latency on warm cache

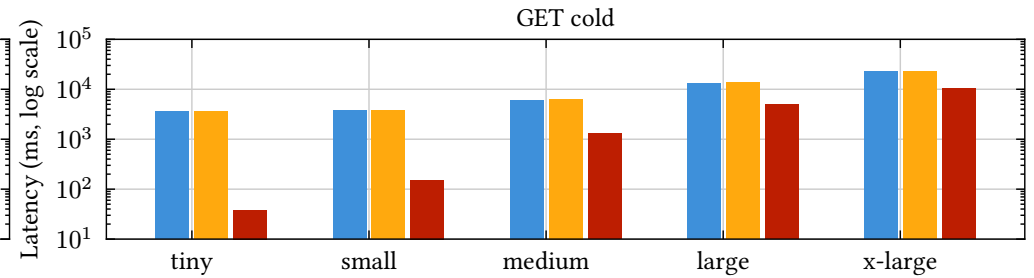


Figure 8b Median GET latency on cold cache

Discussion: Limitations of the Current Design

Core issue: The Parquet files are copied instead of modified

- OSWS does full rewrite of the Parquet file to encrypt and decrypt
- But: different writers produce different files
 - Different metadata, encoding
 - For the client: written file $\neq$ read file
- Result: Inconsistencies in file size
 - Breaks writers that store original size (like DuckLake)

This must be done on **every fetch**

- Range requests: must fetch the entire file first
- Even unauthorized columns copied as dummy data

Ultimately, **PME was not the right fit**

- PME trust model: **client** handles crypto, doesn't trust storage.
- OSWS: **server-side** crypto, client trusts the proxy.

Discussion: Proposed Redesign

1. Internal Metadata Store in the database

- No need to store inside Parquet for OSWS
- Store wrapped DEKs, KEK ref

2. Custom AES-CTR Modifier

- Replace Parquet Sharp entirely with modifier
- Length-preserving → file size never changes
- In-place decryption, no file copy

Question: How to know what key covers a range?

- Idea: Store metadata about what columns cover what range
- Lookup DEKs based on range

Key improvements:

- Files unchanged → addresses DuckLake issue
- Range requests work (fetch + decrypt only needed)
- Async DEK unwrap from pre-fetched wrapped keys

Conclusion

OSWS demonstrates feasibility of FGRBAC in data lakes by encryption

Objectives

1.  Create a PoC of OSWS using an underlying S3-compatible Object Store.
2.  Ensure OSWS enforces access control using RBAC and encryption.
3.  Make it compatible with “fully managed all-in-one cloud platforms”.
4.  Make it compatible with “external query engines”.
5.  Benchmark OSWS to measure the latency it adds.
6.  Reflect on design decisions and which alternatives would have been better.

Correct Design Decisions

- DEK cache
 - Small: $\sim 5.6 \times$ improvement from none to cold
 - Small: $\sim 120 \times$ improvement from cold to warm

Impractical in Current State

- 125MB Parquet file retrieval took 1.6 seconds in best case
- Larger files are even slower

Questions?

Background

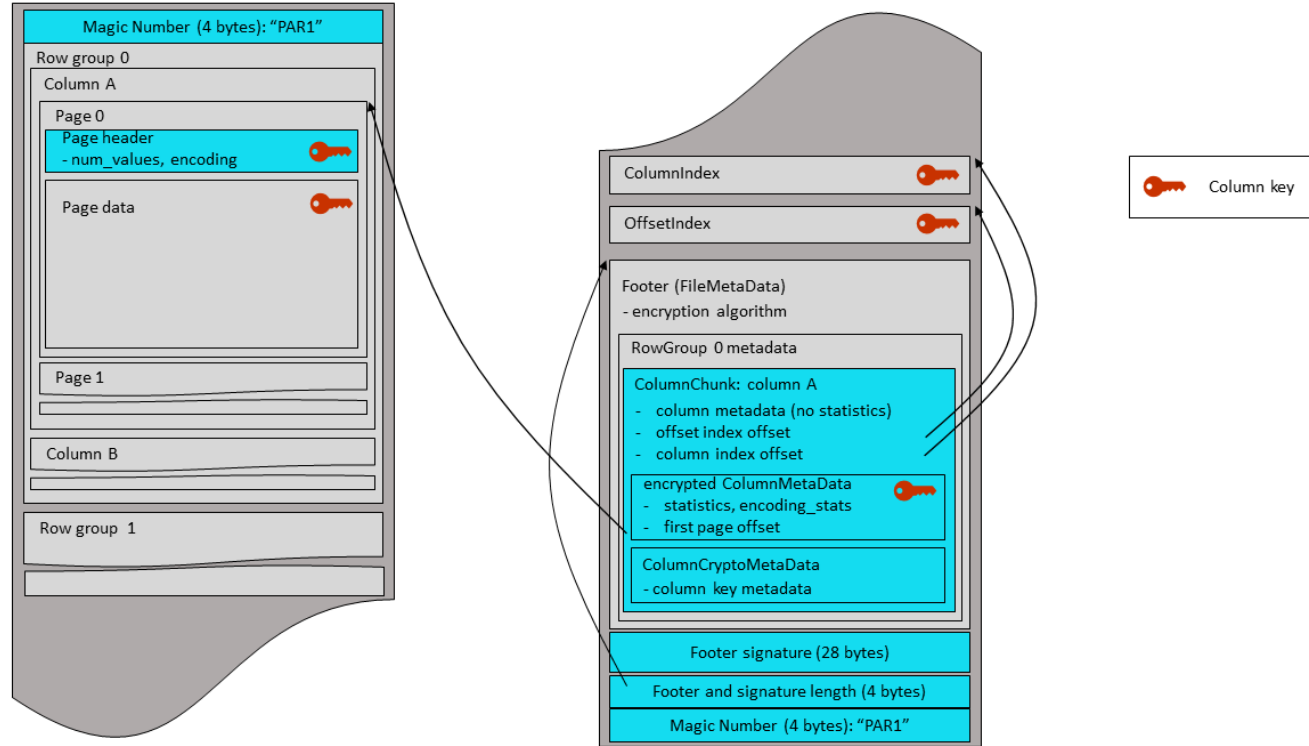


Figure 1

System Design: API

Method	Path	Description
GET	/	List buckets
GET	/ {bucket}	List objects in a bucket
PUT	/ {bucket}	Create bucket
DELETE	/ {bucket}	Delete bucket
POST	/ {bucket}?delete	Delete multiple objects in bucket
HEAD	/ {bucket}	Retrieve bucket metadata
HEAD	/ {bucket}/ {key}	Retrieve object metadata
GET	/bucket}/ {*key}	Retrieve an object (with decryption and column filtering for Parquet)
PUT	/ {bucket}/ {*key}	Store an object (with encryption for Parquet)
DELETE	/ {bucket}/ {*key}	Delete an object

Table II: S3-Compatible endpoints provided by OSWS

Method	Path	Description
GET	/api/me	Current user profile (triggers JIT provisioning)
GET	/api/credentials	List the user's S3 credentials
POST	/api/credentials	Create a new S3 credential (access key + secret key)
DELETE	/api/credentials/{id}	Revoke a credential (soft delete)

Table III: Application endpoints provided by OSWS for web frontend

Method	Path	Description
GET	/api/admin/columns	Get all columns
POST	/api/admin/columns/ {columnId}/roles/ {roleId}	Grant access on column with ID columnId to role with id roleId
DELETE	/api/admin/columns/ {columnId}/roles/ {roleId}	Revoke access on column with ID columnId to role with ID roleId
GET	/api/admin/roles	Get all roles
POST	/api/admin/roles	Create a role
DELETE	/api/admin/roles/{id}	Delete role with ID id
POST	/api/admin/roles/ {parentId}/inherit/ {childId}	Make role with ID parentId inherit role with ID childId
DELETE	/api/admin/roles/ {parentId}/inherit/ {childId}	Delete inheritance of role parentId from role childId
GET	/api/admin/users	Get all users
POST	/api/admin/users/ {userId}/roles/ {roleId}	Grant role with ID roleId to user with ID userId
DELETE	/api/admin/users/ {userId}/roles/ {roleId}	Revoke role with ID roleId from user with ID userId

Table IV: Admin role management endpoints provided by OSWS

Results: Micro-Benchmarks - Roles

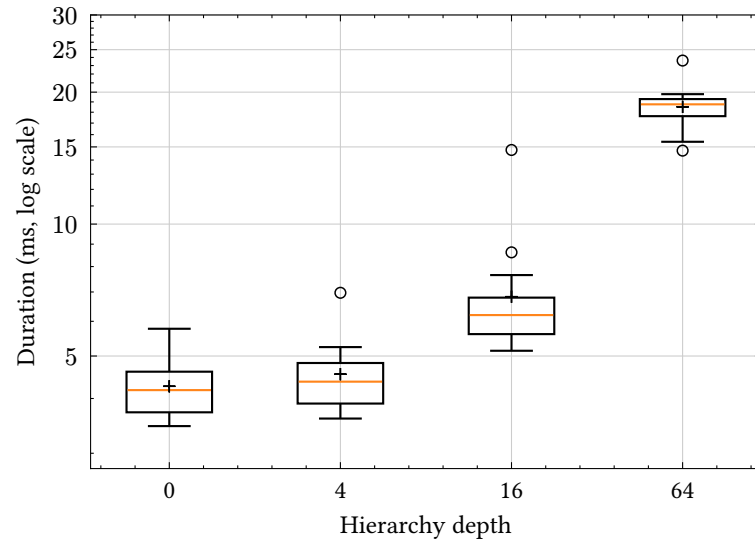


Figure 6c: PermissionHierarchy box-plot distribution per parameter set (logarithmic y axis)

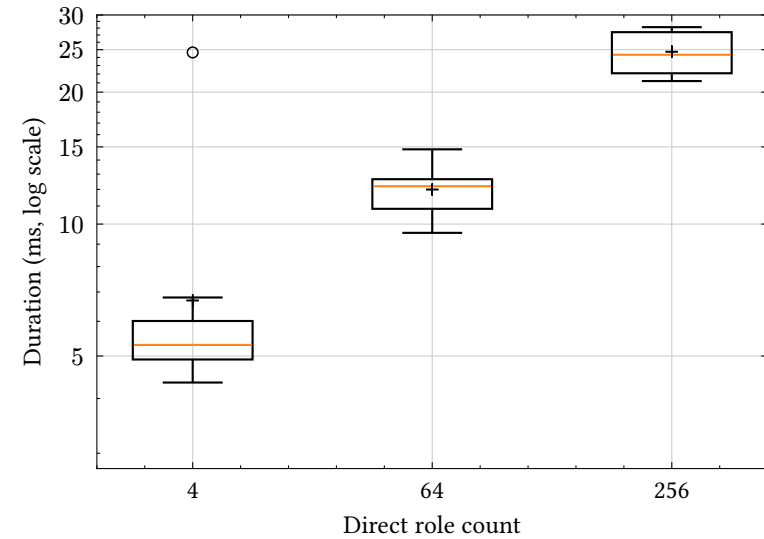
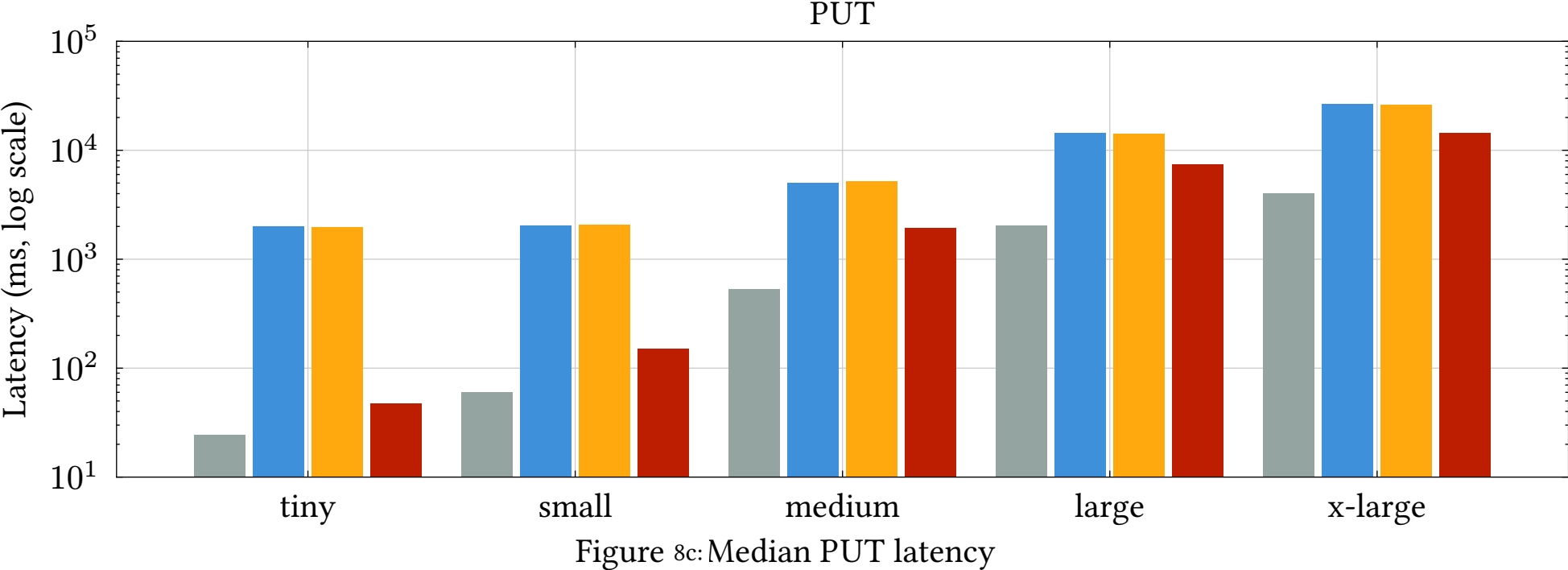


Figure 6d: PermissionService box-plot distribution per parameter set (logarithmic y axis)

Results: E2E Latency - PUT



Discussion: DuckLake error case

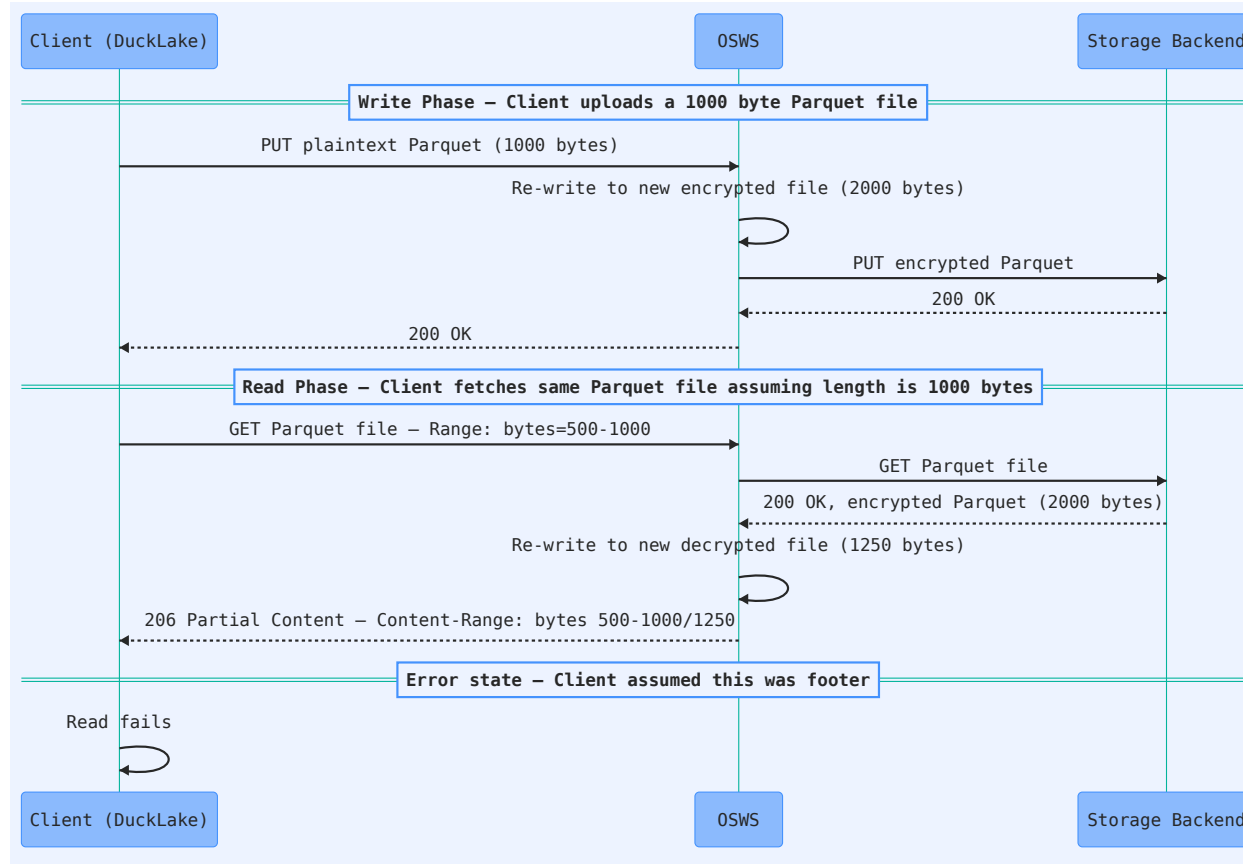


Figure 9: Sequence Diagram showing the DuckLake range request error case. File size and byte ranges are for illustrative purposes only.